IE University

School of Science & Technology

Hierarchical N-Body Simulation of Galactic Dynamics in WebGPU

Enabling Scalable and Interactive Physics Simulations on Modern Web Platforms

Zaid Alsaheb

Bachelor of Computer Science & Artificial Intelligence

Supervised by: Professor Raul Perez Pelaez

1) Literature Review

Accurate simulation of galactic dynamics requires resolving gravitational interactions across large numbers of particles while maintaining numerical stability over long integration times. Achieving this balance has historically required substantial computational resources and specialized software, placing high-fidelity N-body simulations largely outside the reach of lightweight or widely accessible platforms. As a result, the relevant literature spans several traditionally separate domains, including astrophysical N-body methods, numerical integration techniques, GPU-accelerated computing, and browser-based visualization technologies.

This literature review is organized to clarify how these areas intersect and to highlight the technical gap addressed by this work. It first examines the physical and algorithmic foundations of gravitational N-body simulation, with particular attention to the computational scaling challenges that motivate hierarchical approximation methods such as Barnes–Hut. It then surveys prior efforts to accelerate N-body simulations using GPU architectures, with an emphasis on new emergent technologies that allow to achieve native performance in web browsers, drastically enhancing the range of devices that can perform these simulations.

Computational Constraints in Astrophysical Simulation

Over the past decades, progress in computational astrophysics has closely followed the performance improvements predicted by Moore's Law, with central processing unit (CPU) speeds increasing at a near-exponential rate. Once an algorithm was implemented, substantial performance gains could often be achieved simply by running existing code on newer hardware, with minimal additional development effort. However, as single-core CPU performance has plateaued (Sutter, 2005), this implicit scaling model has begun to break down. As a result, continued advances in computational astrophysics increasingly depend on exploiting parallelism and adapting algorithms to emerging computing architectures (Fluke et al., 2011).

One of the most significant architectural shifts has been the widespread adoption of graphics processing units (GPUs), which offer massive parallelism and high memory bandwidth. While GPUs provide the potential for orders-of-magnitude performance improvements (Owens et al., 2007), realizing these gains typically requires substantial algorithmic reformulation. Algorithms that scale well on serial or modestly parallel CPUs may perform poorly on highly parallel architectures if they involve irregular control flow or memory access pattern (Burtscher & Pingali, 2011).

Gravitational N-Body Simulations and Quadratic Scaling

Among the most computationally demanding problems in astrophysics are gravitational N-body simulations, which form the foundation of many studies in galactic dynamics and structure

formation (Binney & Tremaine, 2008). In such simulations, a galaxy is modeled as a system of particles (representing stars or dark matter) whose evolution is governed by their mutual gravitational interactions (Zwart et al., 2007).

According to Newton's law of universal gravitation, the force exerted on a particle of mass m_i by another particle of mass m_j is given by

$$\vec{F}_{ij} = G\frac{m_i m_j(\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|^3} \tag{1}$$

Where $\vec{F}_{ij}$ denotes the gravitational force exerted on particle i by particle j, and G is the gravitational constant. The quantities m_i and m_j represent the masses of particles i and j, respectively. The vectors $\vec{r}_i$ and $\vec{r}_j$ give the positions of particles i and j in three-dimensional space.

The vector difference $\vec{r}_j - \vec{r}_i$ points from particle i toward particle j, while $|\vec{r}_j - \vec{r}_i|$ denotes the Euclidean distance between the two particles. The cubic power of the distance in the denominator ensures that the magnitude of the force follows an inverse-square law while preserving the correct force direction.

The total gravitational force acting on particle i is obtained by summing the pairwise force contributions from all other particles in the system, excluding self-interaction,

$$\vec{F}_i = \sum_{j \neq i}^{N} G\frac{m_i m_j(\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|^3} \tag{2}$$

Evaluating this expression requires computing $N-1$ pairwise interactions per particle. Consequently, a direct implementation of gravitational force evaluation scales as $O(N^2)$ per simulation timestep. This quadratic scaling rapidly becomes computationally prohibitive as the number of particles increases, particularly when long integration times are required to observe large-scale dynamical phenomena such as spiral structure or halo evolution (Zwart et al., 2007).

Hierarchical Approximation and the Barnes–Hut Algorithm

To address the computational limitations imposed by direct force evaluation, a variety of approximation techniques have been developed to reduce the cost of N-body simulations while preserving essential physical behavior. One of the most influential of these methods is the Barnes–Hut algorithm, which exploits the hierarchical spatial structure of particle distributions to approximate gravitational interactions (Barnes & Hut, 1986).

Originally introduced by Barnes and Hut in 1986, the algorithm organizes particles into a tree structure, typically a quadtree in two dimensions or an octree in three dimensions. During force evaluation, distant groups of particles are approximated as a single effective mass located

at their center of mass, allowing subsets of the force summation to be replaced by a single interaction. An opening-angle parameter controls the accuracy of this approximation, providing a tunable tradeoff between computational efficiency and force accuracy.

By replacing many distant pairwise interactions with aggregate approximations, Barnes–Hut reduces the computational complexity of force evaluation from $O(N^2)$ to approximately $O(N \log N)$. This reduction makes the algorithm particularly well-suited for simulating large, collisionless systems such as galaxies, where global dynamical behavior is often of greater interest than exact short-range interactions.

Numerical Integration and Stability Considerations

In addition to force evaluation, the accuracy and long-term stability of N-body simulations depend critically on the numerical integration of the equations of motion. In gravitational N-body problems, the primary quantity of interest is the time evolution of the particle positions, which is governed by Newton's equations of motion. For each particle i, the total gravitational force resulting from interactions with all other particles determines its acceleration according to

$$\sum_j \vec{F}_{ij} = m_i \vec{a}_i \tag{3}$$

Where $\vec{a}_i$ is the acceleration of the particle i. The acceleration is related to the particle velocity and position through time derivatives,

$$\vec{a}_i = \frac{d\vec{v}_i}{dt}, \qquad \vec{v}_i = \frac{d\vec{r}_i}{dt} \tag{4}$$

Because these equations generally cannot be solved analytically for systems containing many interacting particles, their evolution must be approximated numerically. Numerical integration methods advance the particle positions and velocities forward in time using discrete timesteps of size Δt, approximating the continuous dynamics of the system.

One of the simplest numerical integration schemes is the forward Euler method (Kreyszig, 2011). In this approach, particle positions are updated using the current velocity according to

$$\vec{r}_i^{\,n+1} = \vec{r}_i^{\,n} + \vec{v}_i^{\,n} \Delta t \tag{5}$$

Where the superscript n denotes the discrete timestep, with $\vec{r}_i^{\,0}$ representing the initial particle position. Velocities are updated similarly using the acceleration computed from the forces at the current timestep. While the Euler method is computationally inexpensive and straightforward to implement, it suffers from poor numerical stability and does not conserve energy, leading to significant accumulated errors and unphysical behavior over long integration times.

As a result, astrophysical N-body simulations commonly employ symplectic integration schemes such as the Verlet or leapfrog methods, which are specifically designed to preserve the Hamiltonian structure of the equations of motion. These methods exhibit significantly improved long-term energy conservation and are therefore better suited for simulations of galactic dynamics that require stable evolution over many dynamical timescales (Springel, 2005).

In Barnes–Hut simulations, errors introduced by numerical integration interact with approximation errors arising from hierarchical force evaluation (Salmon & Warren, 1994). Prior studies have shown that, for many galactic-scale simulations, integration error can dominate force-approximation error when inappropriate timestepping schemes or integration methods are used (Springel, 2005). Consequently, careful selection of numerical integration techniques remains essential even when approximate force models are employed.

GPU Acceleration of Hierarchical N-Body Methods

The computational demands of gravitational N-body simulations have made them a natural target for graphics processing units (GPUs), which provide massive data parallelism and high memory bandwidth. Modern GPUs are designed with thousands of small, efficient cores capable of executing the same operation on many data elements simultaneously (Fluke et al., 2011). This architecture is particularly well-suited to the independent, particle-wise force computations inherent in N-body simulations (Nyland et al., 2009).

To leverage GPUs effectively, computational tasks are expressed in terms of small programs that run on the GPU cores called shaders. Originally, shaders were designed to compute visual effects for rendering pipelines, including vertex transformations, fragment coloring, and texture operations. Early GPU-based scientific computing therefore relied on repurposing graphics shaders for numerical tasks, encoding computation as rendering operations and storing data in textures (Owens et al., 2008).

Over time, GPU programming languages such as CUDA and OpenCL have generalized this model to allow general-purpose computation, enabling the parallel execution of tasks that are not directly related to graphics, including N-body simulations (Fluke et al., 2011; Nyland et al., 2009).

Despite these advances, hierarchical algorithms such as Barnes–Hut present persistent challenges for GPU architectures. Tree construction and traversal introduce irregular memory access patterns and branch divergence, both of which reduce parallel efficiency on SIMD-style hardware (Burtscher & Pingali, 2011; Karras, 2012). To mitigate these issues, prior work has proposed optimizations including linearized tree representations, stackless traversal methods, and the use of space-filling curves to improve memory coherence (Burtscher & Pingali, 2011;

Gaburov et al., 2010). While these techniques have enabled efficient native GPU implementations, they rely on low-level memory management and flexible data structures that have traditionally been inaccessible in browser-based computing environments.

Since the introduction of the Barnes–Hut algorithm, numerous modifications and alternative hierarchical methods have been proposed to improve accuracy and efficiency, including higher-order multipole expansions and the Fast Multipole Method (FMM) (LF & Rokhlin, 2001; Nyland et al., 2009; Wang, 2021). These approaches differ primarily in how distant particle interactions are approximated and aggregated. Despite these developments, Barnes–Hut remains widely used due to its favorable balance between computational cost, accuracy, and implementation complexity.

General Purpose GPU Computation in the Web

Modern web applications increasingly rely on GPU acceleration to deliver interactive, intelligent, and computation-heavy experiences directly in the browser. Access to GPU resources enables use cases such as real-time large-model reasoning, embedding generation, multimedia processing, and advanced data visualization without requiring users to install specialized software or manage local hardware (Congote et al., 2011; Sung et al., 2025; Usta, 2024). For users working across devices or within managed environments e.g., students, researchers, enterprise teams), browser-based GPU access lowers friction, improves accessibility, and supports scalable, on-demand compute.

WebGL has long served as the primary mechanism for accessing GPU acceleration within web browsers. Standardized by the Khronos Group as a JavaScript binding to OpenGL ES, WebGL was designed to provide portable and efficient access to GPU hardware for real-time graphics rendering across a wide range of devices (Khronos Group, 2024). Its success has enabled a broad ecosystem of browser-based visualization tools, interactive simulations, and educational applications.

Although WebGL exposes programmable shaders, its computational model is fundamentally centered around the graphics rendering pipeline. All computation must be framed in terms of vertex and fragment processing stages, with data represented as textures and intermediate results captured via framebuffers (Congote et al., 2011). As a consequence, general-purpose computation in WebGL requires reformulating numerical algorithms as sequences of rendering passes, often involving non-intuitive data layouts and multiple shader invocations to emulate iteration and state updates.

This graphics-centric abstraction has enabled a class of data-parallel algorithms to be executed efficiently in the browser, particularly those that map naturally onto image-based

representations. However, it imposes significant constraints on algorithms that require irregular memory access, dynamic data structures, or complex control flow (Congote et al., 2011) . In the context of gravitational N-body simulations, these constraints complicate the implementation of adaptive time stepping, hierarchical spatial decomposition, and recursive tree traversal, all of which are central to physically accurate Barnes–Hut simulations (Burtscher & Pingali, 2011).

As a result, prior WebGL-based N-body implementations have typically prioritized real-time performance and visual plausibility over numerical rigor. Common approaches include limiting particle counts, employing softened or approximate force models, reducing integration accuracy, or relying on fixed spatial grids rather than fully adaptive hierarchical trees (Sengupta et al., 2025). While such methods are effective for interactive visualization and outreach, they are not well suited to large-scale galactic simulations where long-term energy conservation and accurate force evaluation are essential.

WebGPU is a modern web standard designed to provide low-level, high-performance access to GPU hardware from within the browser. Unlike WebGL, which is built around a traditional graphics pipeline, WebGPU exposes explicit support for general-purpose GPU computation through compute shaders, storage buffers, and programmable pipelines (GPU for the Web Working Group, 2026). Its design closely mirrors contemporary native graphics and compute APIs such as Vulkan, Metal, and Direct3D 12, enabling fine-grained control over memory layout, synchronization, and parallel execution (Usta, 2024).

Central to WebGPU's computational model is the compute shader: a general-purpose GPU program that executes independently of the rendering pipeline. Compute shaders allow developers to express parallel workloads directly in terms of data processing rather than image generation, making them well suited to scientific computing tasks such as particle simulation, spatial partitioning, and force evaluation (Sung et al., 2025; Usta, 2024). This model enables algorithms to be structured in a manner similar to native GPU implementations, without the need for graphics-specific workarounds (Sengupta et al., 2025).

The explicit memory model and flexible buffer abstractions provided by WebGPU make it possible to represent complex data structures, including hierarchical trees and linearized spatial indices, directly on the GPU (Sung et al., 2025; Usta, 2024). These features are particularly relevant for Barnes–Hut simulations, which rely on dynamic tree construction and traversal to achieve sub-quadratic scaling. While such patterns were previously impractical in web environments, WebGPU enables their implementation with performance characteristics comparable to native GPU code, subject to browser and hardware constraints (Sengupta et al., 2025).

2) Methodology

2.1 Research Design and Objectives

This work adopts a computational-methods design in which a gravitational N-body solver is implemented and evaluated with a focus on reproducibility, long-term numerical stability, and computational scalability beyond the $O(N^2)$ cost of direct summation. The solver is implemented in C++20 using the WebGPU C API and built from a single codebase targeting both native desktop execution (via backends such as wgpu-native and Dawn) and browser execution (compiled via Emscripten) (Khronos Group, 2024). This dual-target approach enables interactive visualization as well as headless batch runs, allowing performance and numerical behavior to be evaluated under comparable conditions across platforms.

The physical and numerical foundations of the implementation draw directly from the literature on hierarchical N-body simulation and GPU parallelism. The gravitational force model adopts Newtonian gravity with Plummer-type softening to avoid the $\frac{1}{r^2}$ singularity and reduce spurious two-body relaxation in collisionless regimes (Binney & Tremaine, 2008). Accelerations are computed using a Barnes–Hut-style hierarchical approximation that reduces force evaluation from $O(N^2)$ to approximately $O(N \log N)$ (Barnes & Hut, 1986), with the primary code path constructing and traversing a Linear Bounding Volume Hierarchy (LBVH) entirely on the GPU using the parallel method of Karras (Karras, 2012). Time integration employs a second-order symplectic leapfrog scheme (kick–drift–kick), selected for its improved long-term energy behavior relative to forward Euler in gravitational systems (Springel, 2005). The compute platform, WebGPU, provides general-purpose parallel compute shaders that enable both native and browser deployment through the same low-level API surface.

The evaluation is structured around three operational research questions:

1. **Scalability**: How does runtime per timestep scale with N for a WebGPU Barnes–Hut implementation compared to a direct $O(N^2)$ baseline at small N?
2. **Numerical quality**: For fixed N, how do timestep size Δt and opening angle θ affect (a) long-term conservation behavior (energy and momentum drift where measurable) and (b) stability under long integrations?
3. **Platform feasibility**: What particle counts and timestep rates are practical under WebGPU constraints (32-bit GPU arithmetic, buffer and memory limits, scheduling overhead, and device variability)?

The choice of WebGPU as the compute platform is central to all three research questions; the following section provides a detailed justification.

2.2 WebGPU Platform Justification

The Barnes–Hut algorithm at $O(N \log N)$ requires a full tree rebuild and traversal at every timestep, each involving operations over the entire particle set. At the particle counts targeted by this work (N up to 10^5), these operations are dominated by data-parallel phases: computing bounding boxes, sorting Morton codes, constructing tree topology, and evaluating forces across all particles simultaneously. This degree of parallelism makes GPU execution essential for interactive frame rates, a conclusion broadly supported by the literature on GPU-accelerated N-body simulation (Fluke et al., 2011; Owens et al., 2007; 2008).

The specific choice of WebGPU over established GPU APIs such as CUDA and OpenCL is motivated by three factors. First, WebGPU is a cross-platform compute and rendering API that exposes general-purpose compute shaders and storage buffers through a unified interface, running natively on Vulkan, Metal, and Direct3D 12 backends while also being deployable in web browsers (GPU for the Web Working Group, 2026; Khronos Group, 2024). This portability is central to the research objective of evaluating N-body simulation across diverse execution environments, from native desktop applications to browser-based deployments. Second, WebGPU's compute shader model provides the primitives required for tree-based algorithms: random-access storage buffer reads and writes, atomic operations for bottom-up aggregation, and flexible workgroup dispatch (Sung et al., 2025; Usta, 2024). Third, WebGPU is an emerging W3C standard with active implementation across major browsers and native runtimes, positioning it as the successor to WebGL for GPU-accelerated web applications (Sengupta et al., 2025).

These advantages are accompanied by constraints that inform the implementation design. GPU-side computation in WebGPU is limited to 32-bit floating-point arithmetic, imposing a precision ceiling on force evaluation and integration. Buffer size limits and memory allocation patterns vary across devices and backends. Scheduling overhead for compute dispatches can be significant relative to kernel execution time at small N, and device variability across integrated and discrete GPUs affects both performance and available features (Sengupta et al., 2025). These constraints are explicitly tested through the platform-feasibility research question.

A comparative assessment of available GPU APIs clarifies the positioning of WebGPU. CUDA provides mature tooling and high performance for tree-based N-body codes (Burtscher & Pingali, 2011; Gaburov et al., 2010), but is locked to NVIDIA hardware and has no browser deployment path. OpenCL offers cross-vendor support on desktop but lacks browser integration and has seen declining adoption in favour of vendor-specific APIs. WebGL provides broad browser reach but was designed for rendering, not general-purpose computation: it

lacks compute shaders, storage buffers, and atomic operations, limiting it to fragment-shader workarounds for GPGPU tasks (Congote et al., 2011). WebGPU is uniquely positioned as both browser-deployable and compute-capable, making it the only viable API for evaluating an N-body solver across native and web environments from a single codebase.

Criterion	**CUDA**	**OpenCL**	**WebGL**	**WebGPU**
Compute shaders	Yes	Yes	No	Yes
Storage buffers	Yes	Yes	No	Yes
Atomic operations	Yes	Yes	No	Yes
Browser deployment	No	No	Yes	Yes
Cross-vendor GPU	No	Yes	Yes	Yes
Native deployment	Yes	Yes	No	Yes
64-bit GPU float	Yes	Varies	No	No

Table 1: Comparison of GPU compute APIs against criteria required for hierarchical N-body simulation. WebGPU is the only API that combines compute shader support with browser deployability.

With the computational platform established, the following sections describe the physical model and numerical methods that operate within it.

2.3 Physical Model and State Representation

2.3.1 Softened gravitational acceleration

Each particle represents a mass element evolving under self-gravity in an isolated (open) domain. Using dimensionless units with $G = 1$, the softened acceleration of particle i is

$$a_i = G \sum_{j \neq i} m_j \frac{r_j - r_i}{\left(\|r_j - r_i\|^2 + \varepsilon^2\right)^{\frac{3}{2}}} \tag{6}$$

The softening parameter ε (default 0.5, configurable) introduces a Plummer-type potential that suppresses the $\frac{1}{r^2}$ singularity at short range (Binney & Tremaine, 2008). This formulation is equivalent to treating each particle as a Plummer sphere rather than a point mass, and the same softening kernel appears in the Plummer-model initial conditions used in Scenario B (see Section 2.9).

2.3.2 GPU-friendly mass packing

To reduce memory bandwidth on the GPU, each particle's mass is stored in the w component of its position vector, yielding a packed `vec4<f32>` layout of (x, y, z, m) that avoids a dedicated mass buffer. Because GPU compute throughput is frequently limited by memory bandwidth rather than arithmetic capacity, this packing halves the number of buffer reads required during force evaluation compared to a separate position and mass layout.

Component	.x	.y	.z	.w
positions[i]	x_i	y_i	z_i	m_i
velocities[i]	$v_{x,i}$	$v_{y,i}$	$v_{z,i}$	0
accelerations[i]	$a_{x,i}$	$a_{y,i}$	$a_{z,i}$	0

Table 2: Particle state layout using vec4<f32> packing. Mass is stored in the w component of the position vector to reduce buffer count and memory bandwidth during force evaluation.

2.3.3 Precision strategy

GPU kernels operate in 32-bit floating point to maximize throughput and match typical WebGPU device capabilities. Diagnostic quantities (total energy and momentum) are computed on the CPU in double precision to reduce accumulation error in long-running integrations. This split reflects the practical precision-versus-performance trade-off inherent in WebGPU environments, where 64-bit GPU arithmetic is not available (Sengupta et al., 2025).

Given the acceleration field defined above, the system state is advanced in time using the integration scheme described in the following section.

2.4 Time Integration

2.4.1 Primary integrator: symplectic leapfrog (KDK)

The primary integration scheme is a fixed-timestep, second-order symplectic leapfrog (kick–drift–kick) (Verlet, 1967). With timestep Δt (default 0.0001), the update sequence is:

1. Half-kick:

$$v_i^{n+1/2} = v_i^n + \frac{\Delta t}{2} a_i^n \tag{7}$$

2. Drift:

$$r_i^{n+1} = r_i^n + \Delta t v_i^{n+1/2} \tag{8}$$

3. Recompute acceleration a_i^{n+1} using updated positions.
4. Half-kick:

$$v_i^{n+1} = v_i^{n+1/2} + \frac{\Delta t}{2} a_i^{n+1} \tag{9}$$

This choice is motivated by the well-known long-term stability advantages of symplectic schemes in gravitational dynamics (Springel, 2005). Unlike forward Euler, which introduces secular energy drift proportional to Δt, the leapfrog scheme is time-reversible and exhibits bounded energy oscillation rather than monotonic growth, making it substantially more suitable for long-horizon integrations in Hamiltonian systems (Binney & Tremaine, 2008). The energy

conservation behavior of leapfrog relative to Euler is quantified in the evaluation protocol (see Section 2.10).

2.4.2 Euler integrator (baseline and fallback)

A forward Euler method is retained as `--integrator euler` to provide a stability baseline. Its update sequence is: tree build, force evaluation, then

$$v \leftarrow v + a\Delta t, \;\; r \leftarrow r + v\Delta t \tag{10}$$

The Euler integrator serves as a lower bound on numerical quality against which the leapfrog scheme is compared.

The most expensive step in each integration cycle is the evaluation of accelerations, addressed through hierarchical approximation in the following section.

2.5 Hierarchical Force Evaluation

2.5.1 Monopole approximation

Hierarchical evaluation approximates distant particle groups by a single monopole at the node's center of mass. For a node with total mass M and center of mass R,

$$a_{i,\text{node}} = GM \frac{R - r_i}{(\|R - r_i\|^2 + \varepsilon^2)^{\frac{3}{2}}} \tag{11}$$

This monopole approximation is used consistently across both tree topologies implemented in this work: a binary Bounding Volume Hierarchy (BVH) on the GPU and an eight-way octree on the CPU. Only the tree representation and the geometry of the opening criterion differ between the two paths.

2.5.2 Opening criteria

An internal node is accepted as a monopole approximation when it is sufficiently small relative to its distance from the target particle. The GPU BVH path uses a tight axis-aligned bounding box (AABB) and tests whether the squared maximum extent of the node satisfies $\frac{\text{maxExtent}^2}{d^2} < \theta^2$, where $\text{maxExtent} = \max(\Delta x, \Delta y, \Delta z)$ is derived from the node's AABB bounds. This extent-based criterion reflects the non-cubic node shapes that arise in a BVH more accurately than a uniform half-width. The CPU octree path uses the conventional half-width criterion $\frac{h^2}{d^2} < \theta^2$, where h is the half-width of the cubic octree cell and $d^2 = \|r_i - R\|^2$. Both formulations are expressed in squared form to avoid a per-node square root. Salmon and Warren provide rigorous error bounds for these opening criteria, showing that the conventional Barnes–Hut criterion can admit unbounded errors for $\theta \geq 1/3$ in pathological configurations (Salmon & Warren, 1994).

The default opening angle $\theta = 0.75$ is used as a practical balance between accuracy and performance. For comparison, GADGET-2 uses θ values in the range 0.5 to 0.7 for cosmological

simulations where higher force accuracy is required (Springel, 2005), while the original Barnes and Hut paper used $\theta = 1.0$ (Barnes & Hut, 1986). The parameter sweeps in the evaluation protocol (see Section 2.10) systematically characterize the accuracy–performance trade-off across $\theta \in \{0.3, 0.5, 0.7, 1.0\}$.

Placeholder: Tree opening criterion geometry showing a target particle, a distant tree node, the distance d, the node extent, and the θ threshold for both BVH (maximum AABB extent) and octree (half-width) variants.

Figure 1: Geometry of the opening criterion. A node is approximated as a monopole when its angular size, as measured by extent/d, falls below the threshold θ. Left: BVH variant using maximum AABB extent. Right: octree variant using cell half-width.

The dual-topology approach motivates a software architecture with multiple execution paths, described in the following section.

2.6 Software Architecture and Execution Modes

The implementation is written in C++20 for host-side orchestration and physics, with WGSL for compute and rendering shaders, using the WebGPU C API directly without wrapper libraries. The build system uses CMake with FetchContent and pinned dependency versions to ensure deterministic builds. Dependencies fetched automatically include WebGPU-distribution (v0.2.0), GLFW (3.4), glfw3webgpu (v1.2.0), spdlog (v1.16.0), Dear ImGui (v1.90.9), and GLM (1.0.2). Three build backends are supported: `WGPU` (wgpu-native), `DAWN` (Dawn), and `EMSCRIPTEN` (browser build with `-sASYNCIFY`, `-sALLOW_MEMORY_GROWTH=1`, `-sUSE_GLFW=3`).

The primary execution mode is GPU-primary: all physics operations (integration, tree construction, and force evaluation) are performed on the GPU each step. Diagnostic quantities are computed on the CPU only at configurable intervals through staging-buffer readback of positions and velocities, with the diagnostic frequency set to every 60 frames in interactive mode and every step or every 50 steps in headless mode depending on N. To support cross-checking and fallback behavior, CPU mirror arrays (`cpuPositions_`, `cpuVelocities_`, `cpuAccelerations_`) are retained and used by non-primary modes (Euler and CPU-tree leapfrog), where scalar CPU loops and a CPU octree are executed in parallel with the GPU path. This design yields two

methodological advantages: first, no per-step CPU physics overhead in the primary mode, since the tree is built directly from GPU-resident particle state; second, cross-validation potential across runs, since selecting the CPU octree versus GPU BVH path provides two independent hierarchical implementations for comparative diagnostics.

Placeholder: System architecture block diagram showing C++ host orchestration, WebGPU compute shader dispatch, GPU storage buffers (positions, velocities, accelerations, BVH nodes), staging buffer readback for diagnostics, CPU mirror arrays for fallback paths, and the rendering pipeline reading directly from storage buffers.

Figure 2: System architecture overview. The GPU-primary path (solid arrows) performs all physics on-device. CPU mirror arrays (dashed arrows) support fallback execution modes and cross-validation. Diagnostic readback occurs at configurable intervals via staging buffers.

The GPU-primary execution mode relies on a set of WebGPU compute shaders organized into a per-timestep pipeline, described in the following section.

2.7 WebGPU Compute Methodology

2.7.1 GPU data layout

Simulation state is stored in WebGPU storage buffers using the packed `vec4<f32>` layout described in Table 2: positions (with mass in the w component), velocities, and accelerations. The BVH is stored as an array of `BVHNode` structures with $2N - 1$ entries (internal nodes indexed 0 to $N - 2$, leaves indexed $N - 1$ to $2N - 2$), where each node stores a center of mass and a tight AABB. A flattened CPU octree buffer is maintained for fallback paths, using explicit child fields (`c0` through `c7`) to avoid dynamic indexing limitations in WGSL. A uniform parameters buffer shares layout-matched simulation parameters between C++ and WGSL. In-place updates are used throughout: there is no double buffering of positions, velocities, or accelerations. Correct ordering between compute passes relies on WebGPU's implicit storage-buffer synchronization between dispatches within a single command buffer submission, with bind groups recreated each frame.

2.7.2 Compute shaders

The implementation comprises twelve WGSL compute shaders organized into two functional groups. The first group handles integration and force evaluation: a direct-summation baseline shader, an octree-traversal fallback shader, the primary BVH-traversal force shader, and kick/drift and Euler integration shaders. The second group implements LBVH construction in seven passes: two-pass bounding-box reduction, Morton code generation (Morton, 1966), bitonic sort with multiple sub-passes (Batcher, 1968), Karras topology construction (Karras, 2012), leaf initialization, and bottom-up aggregation via atomic counters. Workgroup sizes are fixed per kernel (64 for force evaluation, 256 for integration and tree building) and are reported as part of the implementation configuration.

2.7.3 Per-timestep execution sequence (GPU-primary leapfrog)

Each timestep is recorded into a single command encoder and submitted as one command buffer. The six sequential passes are:

1. Half-kick: $v \leftarrow v + \frac{a\Delta t}{2}$
2. Drift: $r \leftarrow r + v\Delta t$
3. LBVH build (7 sub-passes): global AABB reduction → Morton code generation → bitonic sort → Karras topology construction → leaf initialization → bottom-up aggregation
4. BVH force evaluation: iterative traversal with fixed-depth explicit stack (depth 64, sufficient for all tested N)
5. Half-kick: $v \leftarrow v + \frac{a\Delta t}{2}$
6. Diagnostics readback (periodic): stage-map-readback → CPU double-precision energy and momentum computation

Placeholder: Vertical flow diagram showing the six per-timestep passes: half-kick, drift, LBVH build (with 7 sub-passes expanded), BVH force evaluation, half-kick, and periodic diagnostics readback.

Figure 3: Per-timestep GPU execution pipeline for the leapfrog integrator. All six passes are recorded into a single command buffer. The LBVH build (pass 3) comprises seven sub-passes with implicit barrier synchronization between each.

2.7.4 GPU traversal (iterative, no recursion)

Tree traversal is implemented iteratively in the BVH force shader. One GPU thread is assigned per particle. The thread maintains an explicit stack of node indices, beginning from the root. A node is either accepted (leaf or opening criterion satisfied) and accumulated via the monopole approximation, or expanded by pushing its children onto the stack. Fast inverse square root (`inverseSqrt`) is used for inverse-distance evaluation, and a self-interaction guard avoids contributions from degenerate near-zero distances. This approach preserves the Barnes–Hut approximation structure while accommodating GPU execution constraints and limiting branch divergence where possible (Burtscher & Pingali, 2011; Karras, 2012; Nyland et al., 2009).

2.7.5 GPU LBVH construction (Karras 2012)

The LBVH is built fully on-device in seven conceptual steps. First, a two-pass parallel reduction computes the global axis-aligned bounding box. Second, particle positions are normalized to a $[0, 1023]^3$ integer grid and their bits interleaved to produce 30-bit Morton codes (Morton, 1966), which impose a spatial ordering along a space-filling Z-curve. Third, Morton codes and associated particle indices are sorted using a bitonic sort network (Batcher, 1968) with multiple (k, j) sub-passes and dynamic uniform offsets; arrays are padded to the next power of two, with sentinel codes assigned to padding elements. Fourth, the parallel binary tree topology is constructed using the Karras delta function (leading zeros of the XOR of adjacent codes, with tie-breaking for duplicates) (Karras, 2012). Fifth, leaves are initialized by mapping sorted indices to particle positions, masses, and point AABBs. Sixth, internal-node AABBs and

centers of mass are aggregated bottom-up using atomic counters to ensure both children are processed before their parent. The resulting BVH is immediately traversable without CPU-side construction or upload, eliminating a per-step CPU bottleneck in the primary execution mode.

Placeholder: Vertical flow diagram showing the seven LBVH construction sub-passes with data dependencies: (1) global AABB reduction, (2) Morton code generation, (3) bitonic sort, (4) Karras topology, (5) leaf initialization, (6) bottom-up aggregation. Arrows indicate barrier synchronization points between passes.

Figure 4: LBVH construction pipeline. Seven compute dispatches build the tree entirely on-device. Each arrow represents an implicit storage-buffer barrier. The atomic-counter aggregation in pass 6 ensures correct bottom-up propagation of bounding boxes and centers of mass.

2.7.6 CPU octree construction (fallback paths)

The CPU octree is used only by Euler and CPU-tree leapfrog modes. It is built from CPU mirror arrays by computing a bounding box, inserting particles via octant selection, propagating centers of mass bottom-up, and optionally flattening to a GPU-friendly node array when GPU evaluation is used. GPU buffers auto-resize during uploads when needed.

In interactive mode, the GPU-computed particle state is rendered directly from storage buffers, as described in the following section.

2.8 Rendering and Interactive Operation

For visualization, particles are rendered as instanced billboard quads with additive blending. Positions and colors are read directly from storage buffers via `@builtin(instance_index)`, requiring no separate vertex buffer. Depth testing is enabled with depth writes disabled; fragments are masked to a circular footprint with a soft alpha falloff. An ImGui overlay provides interactive control of simulation parameters (θ, Δt, ε, and integrator selection) and displays real-time diagnostics including energy, momentum, and per-component timing breakdowns. In headless mode, the rendering pipeline is skipped entirely and the simulation loop runs without

any presentation overhead, enabling pure throughput measurement. Rendering is a presentation layer and does not alter the simulation state.

Having described the solver and its implementation, the following section specifies the benchmark scenarios used to evaluate it.

2.9 Initial Conditions and Benchmark Scenarios

No external astronomical datasets are used. All experiments are generated from synthetic initial conditions and produce derived outputs (trajectories, diagnostic scalars, and timing logs). This design eliminates licensing and privacy concerns and enables controlled, repeatable comparisons across parameter sweeps. Each experiment is fully specified by command-line parameters: scenario type, seed, N, Δt, θ, softening ε, and step count.

2.9.1 Scenario A: two-body orbit (sanity check)

Scenario A places two equal-mass particles ($m = 1000$ each, $N = 2$ enforced regardless of the `--N` parameter) separated by $d = 10$ units along the x-axis, with tangential velocities along the z-axis computed for a softened circular orbit:

$$v = \sqrt{\frac{Gmd^2}{2(d^2 + \varepsilon^2)^{\frac{3}{2}}}} \tag{12}$$

This configuration provides the simplest possible validation of integrator correctness. With the correct timestep and softening, the two particles should maintain a stable circular orbit indefinitely under the leapfrog scheme. Departures from circularity provide a direct diagnostic for integration error, and the sensitivity to Δt and ε can be isolated without confounding effects from hierarchical force approximation. The scenario is not representative of large-N hierarchical behavior, but it is an essential sanity check that must pass before more complex scenarios are credible.

2.9.2 Scenario B: Plummer sphere (spherical equilibrium test)

Scenario B generates a Plummer sphere (Aarseth et al., 1974; Plummer, 1911) with $N \in [10^3, 10^5]$ (depending on hardware) and scale length $a = 5$. The Plummer model is a classical self-gravitating equilibrium configuration in which the density profile follows $\rho(r) \propto \left(1 + \frac{r^2}{a^2}\right)^{-\frac{5}{2}}$, and it is widely used as a test case for N-body codes because of its known analytic properties (Binney & Tremaine, 2008). Particle radii are sampled via the inverse cumulative distribution function

$$r = \frac{a}{\sqrt{u^{-2/3} - 1}} \tag{13}$$

with u clamped to $[0.001, 0.999]$. Angular coordinates are isotropic: $\cos(\theta)$ is drawn uniformly and the azimuthal angle ϕ is drawn uniformly on $[0, 2\pi)$. Particle speeds are sampled via rejection sampling using

$$g(q) = q^2(1-q^2)^{\frac{7}{2}} \tag{14}$$

against the local escape velocity

$$v_e = \sqrt{\frac{2GM}{\sqrt{r^2+a^2}}} \tag{15}$$

with isotropic velocity directions. This scenario tests tree accuracy and long-term stability in a compact three-dimensional distribution. The Plummer softening parameter ε in the force model is conceptually related to the Plummer scale length a in the initial conditions, though the two are independently configurable. The key sweep variables are θ, ε, and Δt; the primary limitation is that the spherical geometry does not emphasize disk morphology such as spirals and bars.

2.9.3 Scenario C: rotating exponential disk (galaxy-like morphology test)

Scenario C generates a rotating disk with $N \in [10^4, 10^5]$ to evaluate long-term evolution and visually interpretable galactic dynamics. The surface density profile follows an exponential distribution (Freeman, 1970), with particle radii drawn from an exponential distribution (rate parameter 0.08) and clamped to a maximum of 50 units, and azimuthal angles drawn uniformly. Vertical height is drawn from a normal distribution $\mathcal{N}(0, 0.3)$ scaled by $\frac{1}{1+0.5r}$ to produce a thinner disk at larger radii. Particle masses are drawn uniformly from $[0.5, 2.0]$.

Circular velocities are assigned using an approximate enclosed-mass estimate. For particles with $r > 0.1$, the tangential velocity is

$$v = 0.5\sqrt{\frac{M_{\text{enclosed}}}{r}} \tag{16}$$

with the velocity directed tangentially. This simplified dynamical setup is not a full multi-component Milky Way model: the enclosed-mass estimate is approximate, and no bulge or halo component is included. Nevertheless, the disk geometry provides a morphologically rich test case in which the formation and persistence of spiral structure, bars, and other large-scale features serve as qualitative validation of the solver's long-term behavior. The key sweep variables are disk scale length, thickness, velocity dispersion, θ, and Δt.

2.9.4 Sampling and robustness across seeds

Because initial conditions are stochastic, robustness is assessed by repeating runs with different seeds (`--seed 1`, `--seed 2`, and so on) and comparing diagnostics and timing. Initial condition generation uses `std::mt19937` seeded by the `--seed` parameter (default 42), ensuring deter-

ministic reproduction of particle distributions and velocities. Runs are considered valid if they complete without NaN or overflow values and produce consistent parameter logs. Deliberately unstable configurations (such as excessively large Δt) are retained as documented failures for robustness reporting rather than silently excluded.

Placeholder: Three-panel figure showing initial particle distributions. Left: Scenario A (two-body orbit, two particles with velocity vectors). Center: Scenario B (Plummer sphere, spherically symmetric particle cloud). Right: Scenario C (exponential disk, face-on view showing disk structure).

Figure 5: Initial particle distributions for the three benchmark scenarios. (a) Scenario A: two-body orbit. (b) Scenario B: Plummer sphere with $N = 10000$. (c) Scenario C: exponential disk with $N = 50000$.

The evaluation protocol systematically exercises these scenarios across the parameter space defined above.

2.10 Evaluation Protocol: Baselines, Metrics, and Parameter Sweeps

2.10.1 Baselines

Two baselines are used. The first is a direct $O(N^2)$ summation, which serves as a reference computation path for small N. Potential energy is computed by direct pair summation only when $N \leq 5000$ due to its $O(N^2)$ cost. The second baseline is the forward Euler integrator (`--integrator euler`), which provides a numerical stability reference against which the leapfrog scheme is compared.

2.10.2 Primary metrics

Runtime per timestep (measured in milliseconds per step using `std::chrono::high_resolution_clock`) is decomposed into three components: tree build time (GPU LBVH construction or CPU octree construction plus upload), force evaluation time (GPU BVH traversal or CPU Barnes–Hut walk), and integration time (kick and drift dispatches, including any CPU mirror loops on fallback paths). This decomposition identifies which phase dominates at each particle count and reveals the tree-build fraction, which is a key indicator of the overhead introduced by per-step hierarchy construction.

Scaling with particle count is characterized by measuring total and per-component runtimes across a range of N values for both hierarchical and direct modes, producing empirical scaling curves that are compared against the expected $O(N \log N)$ and $O(N^2)$ asymptotic behavior. Long-term stability is assessed through energy drift, defined as $\Delta E(t) = |E(t) - E(0)|/|E(0)|$, reported only for $N \leq 5000$ where potential energy is computed via direct pair summation. For larger N, stability is assessed through kinetic energy trends and momentum diagnostics.

2.10.3 Secondary metrics

Linear momentum magnitude $\|P(t)\| = \|\sum_i m_i v_i\|$ is computed in double precision and is expected to remain near zero for symmetric initial conditions (Plummer sphere) while reflecting the net angular momentum of the disk scenario. Qualitative morphology in disk runs (persistence of spiral arms, bar formation, and overall structural evolution) is reported descriptively as a complement to quantitative diagnostics, providing a visual sanity check on solver behavior over long integration times.

2.10.4 Parameter sweeps

To characterize accuracy–performance trade-offs and stability regimes, controlled parameter sweeps are performed via the command-line interface. The opening angle θ is swept over $\{0.3, 0.5, 0.7, 1.0\}$, the timestep Δt is varied across a scenario-dependent stable range, and the softening parameter ε is tested across representative values. Each sweep also compares the Euler and leapfrog integrators and the CPU octree versus GPU LBVH construction paths. Results are compared using exported CSV logs containing per-step diagnostics and timing.

2.10.5 Error analysis procedure

When instability or anomalous drift occurs, runs are inspected for correlations with dense versus diffuse particle regions, large θ, large Δt, small ε, and platform or backend differences. Failures (NaN values, overflow, and extreme velocities) are detected explicitly and logged rather than silently suppressed.

2.10.6 Comparative positioning

To situate the performance of this implementation within the broader landscape of GPU N-body codes, key metrics are mapped to those reported in existing literature. Burtscher and Pingali report throughput for a CUDA Barnes–Hut implementation on NVIDIA hardware (Burtscher & Pingali, 2011), and Gaburov, Bédorf, and Portegies Zwart report CUDA tree-code performance including tree-build fractions at varying N (Gaburov et al., 2010). Nyland, Harris, and Prins provide reference throughput figures for GPU N-body methods in the CUDA ecosystem (Nyland et al., 2009). While direct comparison is limited by differences in hardware generation, precision (64-bit versus 32-bit), tree topology (octree versus BVH), and opening criteria, the

metrics collected in this work (throughput at given N, tree-build fraction of total step time, and crossover N between direct and hierarchical methods) are chosen to enable meaningful comparison with these published results.

Beyond the quantitative metrics, the methodology incorporates several validation strategies to ensure correctness and robustness.

2.11 Validation and Robustness

The validation strategy rests on three pillars: theoretical grounding, empirical checks, and explicit acknowledgment of practical constraints. On the theoretical side, the Barnes–Hut hierarchical approximation with tunable opening angle θ follows Barnes and Hut (Barnes & Hut, 1986), the leapfrog integration scheme is a standard choice for long-horizon gravitational dynamics due to its symplectic stability properties (Springel, 2005), the GPU hierarchy construction follows the widely used parallel LBVH method of Karras (Karras, 2012), and the GPU traversal methodology follows established considerations for irregular tree algorithms on SIMD-style hardware (Burtscher & Pingali, 2011; Nyland et al., 2009).

Empirical validation is provided at multiple levels. Scenario A (two-body orbit) offers a controlled sanity check for orbit stability and integrator correctness, where deviations from the analytic circular orbit can be directly measured. Deterministic seeds and complete parameter logging ensure that any run can be exactly reproduced. Fixed step counts (`--steps`) provide consistent comparison across runs, and instability events are recorded rather than filtered. The availability of both CPU octree and GPU BVH execution paths allows cross-validation of force evaluation results for the same initial conditions.

Practical constraints are explicitly acknowledged. Interactive runs couple stepping to the render loop, while headless mode prioritizes throughput. Particle count is adjustable from 2 to 100,000. Potential energy tracking is intentionally limited to $N \leq 5000$ to avoid prohibitive $O(N^2)$ overhead; for larger N, stability is assessed through kinetic energy and momentum diagnostics combined with qualitative morphological assessment. These precision and buffer constraints connect directly to the WebGPU platform limitations described in the platform justification section, where 32-bit arithmetic and device variability were identified as fundamental characteristics of the target environment.

2.12 Data Ethics, Security, and Integrity

No personal or sensitive data are collected. All computation runs locally (native) or within the user's browser sandbox (Emscripten). Diagnostic logs and trajectories are exported only when explicitly requested (`--export`). Each CSV export is linked to the complete set of runtime

parameters (scenario, N, Δt, θ, ε, seed, integrator, steps). The primary practical risk is high GPU load; mitigation is provided through adjustable N and configurable step limits.

2.13 Reproducibility and Traceability

Reproducibility of build and configuration is ensured through several mechanisms. Initial condition generation uses `std::mt19937` with the seed recorded at startup (default 42), producing deterministic particle distributions and velocities for any given parameter set. All third-party dependency versions are pinned through CMake FetchContent, and the complete set of CLI parameters is logged at startup via spdlog alongside the GPU adapter name and selected WebGPU backend, creating a full record of the execution environment for each run.

Output reproducibility and cross-environment comparison are supported by the CSV export format, which records per-step diagnostics and timing: step number, simulation time, kinetic energy, potential energy (when $N \leq 5000$), total energy, energy drift, momentum components (p_x, p_y, p_z), tree-build time, force-evaluation time, and integration time. The dual build targets (native and browser) are produced from the same source code, enabling direct cross-environment comparison of both numerical results and performance characteristics under identical simulation parameters.

3) Experiments

This section specifies the hardware environment and the concrete experiment configurations used to evaluate the solver against the three research questions defined in the methodology. All experiments use the headless batch mode of the simulation, eliminating rendering overhead and producing deterministic CSV logs for post-processing.

3.1 Experimental Platform

All experiments were executed on a single workstation whose configuration is summarised in Table 3. The GPU adapter name and WebGPU backend are reported by the application at startup and recorded alongside each run.

Component	**Specification**
GPU	Apple M2 (unified memory, Metal backend via wgpu-native)
CPU	Apple M2 (8-core, 4 performance + 4 efficiency)
RAM	8 GB unified memory
Operating System	macOS (Darwin 25.2.0)
WebGPU Backend	wgpu-native (Rust-based, Metal backend 0x5)
Compiler	Apple Clang, C++20, Release build (-O2)
Build System	CMake + FetchContent (pinned dependency versions)
Timing	`std::chrono::high_resolution_clock` (ms precision)
Diagnostic Precision	CPU double-precision (64-bit) for energy and momentum

Table 3: Hardware and software configuration for all experiments. The Apple M2 integrated GPU uses Metal as the underlying graphics API, accessed through the wgpu-native WebGPU implementation.

Each run is invoked from the command line as:

```
./galaxysim --headless --scenario <S> --N <N> --dt <dt>
  --theta <θ> --softening <ε> --integrator <I>
  --tree <T> --force-method <F> --steps <steps>
  --seed <seed> --export <output.csv>
```

The CSV output records twelve columns per step: step number, simulation time, kinetic energy, potential energy (zero when $N > 5000$), total energy, energy drift, three momentum components (p_x, p_y, p_z), and three timing components (tree build, force evaluation, and integration in milliseconds). Timing values are averaged over all steps after discarding the first ten as warmup.

3.2 Experiment Groups

The experiments are organised into five groups, each targeting one or more research questions. Table 4 provides a summary.

Group	Scenario	Variable	Runs	RQ
1: Two-body validation	A (two-body)	$\Delta t \times$ integrator	10	RQ2
2a: N-scaling	B (Plummer)	$N \in \{100\text{–}10^5\}$	9	RQ1, RQ3
2b: Theta sweep	B (Plummer)	$\theta \in \{0.3, 0.5, 0.7, 1.0\}$	4	RQ2
2c: Timestep sweep	B (Plummer)	$\Delta t \in \{5 \times 10^{-5}\text{–}5 \times 10^{-3}\}$	5	RQ2
2d: Integrator comparison	B (Plummer)	Euler vs leapfrog	1	RQ2
2e: Softening sweep	B (Plummer)	$\varepsilon \in \{0.1\text{–}2.0\}$	5	RQ2
2f: Seed robustness	B (Plummer)	seed $\in \{42, 123, 256\}$	3	RQ3
3a: Disk N-scaling	C (disk)	$N \in \{10^4\text{–}10^5\}$	5	RQ1, RQ3
4: Direct vs tree crossover	B (Plummer)	force method $\times N$	12	RQ1
5: Native vs browser	B (Plummer)	execution platform $\times N$	9	RQ3

Table 4: Summary of experiment groups, swept variables, and research questions addressed. A total of 63 individual runs were executed.

3.2.1 Group 1: Two-Body Orbit Validation (Scenario A)

This group verifies integrator correctness using the two-body orbit configuration ($N = 2$, $m = 1000$ each). The timestep Δt is swept over $\{0.0001, 0.0005, 0.001, 0.005, 0.01\}$ for both the leapfrog and Euler integrators, yielding ten runs of 50,000 steps each. All runs use the GPU BVH tree path with $\theta = 0.75$, $\varepsilon = 0.5$, and seed 42. The primary diagnostic is the energy drift $\Delta \frac{E(t)}{|} E(0)|$ over the full integration, with secondary attention to momentum magnitude and orbit stability.

3.2.2 Group 2: Plummer Sphere Parameter Sweeps (Scenario B)

The Plummer sphere scenario provides the primary quantitative testbed because it has symmetric initial conditions and known analytic equilibrium properties. Six sub-groups isolate individual parameter effects.

Sub-group 2a (N-scaling) varies N over $\{100, 500, 1000, 2000, 5000, 10000, 25000, 50000, 100000\}$ with all other parameters fixed at defaults ($\Delta t = 0.001$, $\theta = 0.75$, $\varepsilon = 0.5$, leapfrog, GPU LBVH). Each run executes 1,000 steps. This sub-group directly addresses RQ1 (scalability) and RQ3 (feasibility at large N).

Sub-groups 2b through 2e isolate the effects of individual parameters at fixed $N = 5000$ (chosen because potential energy is computed at this size, enabling full energy tracking). Sub-group 2b sweeps $\theta \in \{0.3, 0.5, 0.7, 1.0\}$; sub-group 2c sweeps $\Delta t \in \{5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}\}$; sub-group 2d compares Euler to leapfrog at $\Delta t = 0.001$; and sub-group 2e sweeps $\varepsilon \in \{0.1, 0.25, 0.5, 1.0, 2.0\}$. All parameter-sweep runs execute 5,000 steps.

Sub-group 2f repeats the default configuration ($N = 5000$, $\Delta t = 0.001$, $\theta = 0.75$) with three seeds (42, 123, 256) for 1,000 steps to assess timing and diagnostic variability across stochastic initial conditions.

3.2.3 Group 3: Rotating Disk Scaling (Scenario C)

The disk scenario targets large-N scalability and qualitative morphological assessment. Sub-group 3a scales N over $\{10000, 25000, 50000, 75000, 100000\}$ with default parameters and 1,000 steps each, measuring runtime scaling and timing decomposition. Morphological evolution (spiral arm formation, bar instability) is assessed through interactive-mode visualisation at selected timesteps and reported descriptively.

Placeholder: Four-panel figure showing disk morphology evolution at $t = 0$, $t = 1$, $t = 5$, and $t = 10$ simulation time units for Scenario C with $N = 50000$. Capture from interactive visualisation mode.

Figure 6: Morphological evolution of the rotating exponential disk (Scenario C, $N = 50000$). Panels show the face-on particle distribution at four simulation times, illustrating the development of spiral structure.

3.2.4 Group 4: Direct vs Tree Crossover

To identify the particle count at which hierarchical force evaluation becomes faster than direct $O(N^2)$ summation, this group runs both `--force-method direct` and `--force-method tree` at $N \in \{100, 200, 500, 1000, 2000, 5000\}$ using the Plummer sphere scenario. Each configuration runs for 500 steps with default physics parameters.

3.2.5 Group 5: Native vs Browser Execution

To evaluate WebGPU's portability promise (RQ3), the same GPU LBVH Barnes–Hut configuration used in Group 2a is executed in a browser environment via Emscripten. The native C++/wgpu-native binary is cross-compiled with Emscripten using `-sASYNCIFY` (to yield to the browser event loop each timestep) and `-sALLOW_MEMORY_GROWTH=1`. The resulting WebAssembly module runs in a headless Chromium instance on the same hardware, using the browser's WebGPU implementation (backed by the same Metal driver). Particle counts sweep $N \in$

$\{100, 500, 1000, 2000, 5000, 10000, 25000, 50000, 100000\}$ with all other parameters matching Group 2a ($\Delta t = 0.001$, $\theta = 0.75$, $\varepsilon = 0.5$, leapfrog, GPU LBVH, 1000 steps).

Two timing metrics are collected: (1) wall-clock milliseconds per step, computed from the console-log timestamps emitted every 100 steps, which captures all overhead including event-loop scheduling and Emscripten asyncify yields; and (2) GPU-side timing from the sampled per-step tree/force/integrate breakdowns logged to the console. Energy drift at the final step is compared to the native run at each N to verify numerical consistency across platforms. A GPU command-buffer flush was required after each compute dispatch in the browser path to prevent command coalescing from stalling the pipeline; this fix had no effect on native execution.

4) Results and Analysis

This section presents the empirical results from the experiment groups defined in the previous section, organised by research question. All timing values are mean milliseconds per step computed after discarding the first ten steps as warmup. Energy drift is reported as the final-step value $\Delta E = |E(t) - E(0)|/|E(0)|$.

4.1 Performance Overview

Table 5 condenses the key metrics across representative configurations. The following subsections examine each research question in detail.

Configuration	N	ms/step	Tree build	Force	Integrate
Plummer, GPU tree	1,000	1.04	0.78	0.22	0.05
Plummer, GPU tree	5,000	1.34	1.02	0.27	0.06
Plummer, GPU tree	10,000	1.78	1.38	0.37	0.04
Plummer, GPU tree	50,000	2.20	1.73	0.43	0.04
Plummer, GPU tree	100,000	2.35	1.85	0.46	0.04
Disk, GPU tree	50,000	1.72	—	—	—
Disk, GPU tree	100,000	2.04	—	—	—
Plummer, CPU tree	50,000	65.06	—	—	—
Plummer, direct	5,000	4.96	—	—	—

Table 5: Performance summary for representative configurations (leapfrog, $\theta = 0.75$, $\varepsilon = 0.5$, $\Delta t = 0.001$). Timing components in milliseconds.

4.2 RQ1: Scalability

4.2.1 Runtime Scaling with Particle Count

Figure 7 presents the mean runtime per timestep as a function of N for the GPU LBVH Barnes–Hut path on the Plummer sphere scenario. Total runtime per step increases from 0.61 ms at $N = 100$ to 2.35 ms at $N = 100000$, representing a factor of 3.9 increase over a 1000-fold growth in particle count. This sub-linear scaling is consistent with $O(N \log N)$ force evaluation overlaid with GPU parallelism: the arithmetic cost grows as $N \log N$, but the GPU's data-parallel execution absorbs much of this growth up to the point where occupancy saturates.

Placeholder: Log-log plot of mean ms/step vs N for Plummer sphere (GPU LBVH, leapfrog). X-axis: N from 100 to 100,000. Y-axis: ms/step from 0.5 to 3. Include $O(N \log N)$ reference line. Data points: (100, 0.61), (500, 0.88), (1000, 1.04), (2000, 1.22), (5000, 1.34), (10000, 1.78), (25000, 2.00), (50000, 2.20), (100000, 2.35).

Figure 7: Mean runtime per timestep as a function of N for the Plummer sphere scenario (GPU LBVH, leapfrog, $\theta = 0.75$). The sub-linear growth is consistent with GPU-parallel $O(N \log N)$ force evaluation.

The rotating disk scenario exhibits similar behaviour. Runtime increases from 1.73 ms at $N = 10000$ to 2.04 ms at $N = 100000$, with a slight non-monotonicity at $N = 50000$ (1.72 ms) likely attributable to workgroup occupancy effects on the Apple M2 GPU.

4.2.2 Tree vs Direct Crossover

Table 6 presents the comparison between hierarchical ($O(N \log N)$) and direct ($O(N^2)$) force evaluation. At $N = 1000$ and below, direct summation is faster because the per-step tree construction overhead exceeds the savings from reduced force evaluations. At $N = 2000$, the tree path becomes faster (1.15 ms versus 2.34 ms), and the advantage grows with N: at $N = 5000$, the tree path is 3.5 times faster. The crossover occurs between $N = 1000$ and $N = 2000$, consistent with the expectation that tree overhead is amortised only when N is large enough for the $O(N^2)$ to $O(N \log N)$ reduction to dominate.

N	Direct (ms)	Tree (ms)	Faster
100	0.21	0.83	Direct
200	0.29	0.98	Direct
500	0.65	1.21	Direct
1,000	1.14	1.25	Direct
2,000	2.34	1.15	Tree
5,000	4.96	1.43	Tree

Table 6: Mean ms/step for direct $O(N^2)$ versus tree-based $O(N \log N)$ force evaluation (Plummer sphere, GPU, leapfrog). The crossover occurs between $N = 1000$ and $N = 2000$.

Placeholder: Log-log plot showing direct ms/step (orange) and tree ms/step (blue) vs N. Lines cross between N=1000 and N=2000. Include data from the table above.

Figure 8: Direct vs tree force evaluation runtime. The crossover at $N \approx 1500$ marks where the per-step tree construction cost is amortised by the reduction in force evaluations.

4.2.3 Timing Decomposition

Table 7 breaks total runtime into its three components. Tree construction dominates at all tested particle counts, accounting for 74 to 79 percent of total step time. Force evaluation accounts for 18 to 20 percent, and integration (kick and drift dispatches) accounts for approximately 2 to 4 percent. The dominance of tree construction reflects the cost of the seven-pass LBVH pipeline (bounding box reduction, Morton code generation, bitonic sort, Karras topology, leaf initialisation, bottom-up aggregation) relative to the single-pass BVH traversal for force evaluation.

N	Tree build (ms)	Force (ms)	Integrate (ms)	Tree fraction
1,000	0.78	0.22	0.05	74.3%
5,000	1.02	0.27	0.06	75.6%
10,000	1.38	0.37	0.04	77.2%
50,000	1.73	0.43	0.04	78.5%
100,000	1.85	0.46	0.04	78.6%

Table 7: Timing decomposition for the GPU LBVH leapfrog path (Plummer sphere, $\theta = 0.75$). Tree construction accounts for 74–79% of total step time.

Placeholder: Stacked bar chart showing tree_build_ms (blue), force_ms (orange), integrate_ms (green) at N = 1000, 5000, 10000, 50000, 100000. Tree build is the dominant component at all N.

Figure 9: Stacked timing decomposition at selected particle counts. Tree construction (blue) dominates at all N, with force evaluation (orange) as the secondary component.

4.3 RQ2: Numerical Quality

4.3.1 Two-Body Orbit Validation (Scenario A)

The two-body orbit provides the simplest test of integrator correctness. Table 8 presents the final energy drift after 50,000 steps for both integrators across the timestep sweep. A notable observation is that the Euler integrator, run on the CPU octree path, exhibits substantially lower energy drift (on the order of 10^{-5} to 10^{-6}) than the leapfrog integrator on the GPU BVH path (drift of order unity). This counterintuitive result is attributable to the different force computation paths rather than the integration scheme: the CPU path computes forces in double precision via the octree, while the GPU BVH path uses 32-bit floating-point arithmetic. At $N = 2$, the BVH tree structure is degenerate (a single internal node with two leaves), and the 32-bit force evaluation accumulates sufficient rounding error over 50,000 steps to produce measurable orbit drift.

Δt	Leapfrog (GPU BVH)	Euler (CPU octree)
0.0001	1.71	1.78×10^{-6}
0.0005	1.94	1.08×10^{-5}
0.001	1.97	1.89×10^{-5}
0.005	1.99	1.12×10^{-5}
0.01	1.99	3.81×10^{-6}

Table 8: Final energy drift $\Delta\frac{E}{|}E(0)|$ after 50,000 steps for the two-body orbit (Scenario A). The difference between integrators is dominated by the force computation precision: GPU BVH (32-bit) versus CPU octree (64-bit).

This result highlights that for very small N, the 32-bit precision of GPU computation is the binding constraint on numerical quality, not the integration scheme. The two-body scenario

remains useful as a code-path sanity check (both paths complete without NaN or divergence), but quantitative energy-conservation comparisons between integrators require the same force computation path.

Placeholder: Screenshot of two-body orbit from interactive mode showing the circular orbit path of the two particles. Capture at $t \approx 5$ with trail rendering if available.

Figure 10: Two-body orbit trajectory (Scenario A) as rendered in interactive visualisation mode. The two equal-mass particles maintain a stable circular orbit under the leapfrog integrator.

4.3.2 Energy Conservation in the Plummer Sphere (Scenario B)

The Plummer sphere at $N = 5000$ (where potential energy is computed via direct pair summation) provides the primary testbed for energy conservation. Figure 11 shows the evolution of energy drift over 5,000 steps.

The leapfrog integrator at default parameters ($\Delta t = 0.001$, $\theta = 0.75$) produces a final drift of $\Delta \frac{E}{\mid} E(0)| = 1.84$ after 5,000 steps (simulation time $t = 5.0$). Inspection of the energy components reveals that kinetic energy remains nearly constant ($K \approx 744233$) while the potential energy magnitude decreases steadily from -1.47×10^6 to -5.72×10^5, indicating that the Plummer sphere is expanding. This behaviour is characteristic of a system that is not in perfect virial equilibrium at the discrete N used, combined with the systematic force error introduced by the monopole approximation at $\theta = 0.75$. The total energy transitions from negative to positive, reflecting an unbinding process that is physical in the context of finite-N sampling with approximate forces.

The Euler integrator at the same N and Δt produces a substantially lower final drift of 9.28×10^{-3}, again attributable to the 64-bit CPU force computation path rather than intrinsic integrator quality.

Placeholder: Line plot of energy drift vs step number for Plummer sphere N=5000. Show leapfrog (GPU, blue) rising to 1.84 and Euler (CPU, orange dashed) staying near 0.01. X-axis: 0 to 5000 steps. Y-axis: energy drift (log scale or linear).

Figure 11: Energy drift evolution for the Plummer sphere ($N = 5000$, $\Delta t = 0.001$, $\theta = 0.75$). The difference between integrators reflects the underlying force computation precision (32-bit GPU vs 64-bit CPU) rather than integration scheme quality.

4.3.3 Effect of Opening Angle θ

Table 9 presents the effect of the opening angle on both runtime and energy drift at $N = 5000$. Surprisingly, the final energy drift is nearly identical across all tested θ values ($\Delta \frac{E}{|} E(0)| \approx 1.84$), suggesting that at this N and simulation duration, the dominant source of energy non-conservation is not the multipole approximation error but rather the 32-bit force precision and finite-N relaxation effects. Runtime is also stable across θ values (3.5 to 3.7 ms/step), indicating that at $N = 5000$ the traversal depth does not vary substantially with θ on this hardware.

θ	Mean ms/step	Final drift
0.3	3.46	1.84
0.5	3.65	1.84
0.7	3.66	1.84
1.0	3.49	1.84

Table 9: Effect of opening angle θ on runtime and energy drift (Plummer sphere, $N = 5000$, $\Delta t = 0.001$, leapfrog, GPU BVH). Energy drift is insensitive to θ at this configuration, suggesting 32-bit precision dominates.

4.3.4 Effect of Timestep Δt

The timestep sweep at $N = 5000$ reveals that energy drift increases with simulation time. Because all runs execute 5,000 steps, smaller Δt corresponds to shorter total simulation time: $\Delta t = 5 \times 10^{-5}$ yields $t_{\text{final}} = 0.25$, while $\Delta t = 5 \times 10^{-3}$ yields $t_{\text{final}} = 25$. The drift increases from 0.38 at $\Delta t = 5 \times 10^{-5}$ to 1.99 at $\Delta t = 5 \times 10^{-3}$, reflecting both longer physical evolution time and larger per-step truncation error. To isolate the effect of Δt on integration accuracy

from the effect of total simulation time would require runs to a fixed t_{final}, which was not performed in this sweep and represents an area for refined experimentation.

4.3.5 Effect of Softening ε

The softening sweep at $N = 5000$ shows a modest increase in energy drift with larger ε: from $\Delta \frac{E}{|} E(0)| = 1.82$ at $\varepsilon = 0.1$ to 2.00 at $\varepsilon = 2.0$. Larger softening reduces the depth of the gravitational potential well, making the system less tightly bound and more prone to expansion. Runtime is unaffected by softening (approximately 3.5 ms/step for all values), confirming that softening does not change the computational cost of force evaluation.

4.3.6 Momentum Conservation

For the Plummer sphere at $N = 5000$, the total momentum magnitude $\|P(t)\| = \|\left(\sum_i m_i v_i\right)\|$ remains approximately constant throughout the integration, starting at $\|P(0)\| \approx 1400$ and varying by less than 0.1% over 5,000 steps. This near-conservation is expected for the leapfrog scheme, which preserves linear momentum exactly for pairwise central forces. The non-zero initial momentum arises from the finite-N sampling of the Plummer distribution, which does not enforce exact momentum balance.

4.4 RQ3: Platform Feasibility

4.4.1 Native vs Browser Execution

Table 10 presents the comparison between native desktop execution (wgpu-native, Metal backend) and browser execution (Emscripten WebAssembly, browser WebGPU) for the same GPU LBVH Barnes–Hut configuration. The wall-clock time per step in the browser is remarkably constant at 5.5–6.3 ms regardless of N, while native execution scales from 0.61 ms at $N = 100$ to 2.35 ms at $N = 100000$. The overhead factor therefore decreases from $9.2\times$ at $N = 100$ to $2.7\times$ at $N = 100000$.

N	Native (ms)	Browser (ms)	Overhead	Native drift	Browser drift
100	0.61	5.62	9.2×	1.85×10^{-1}	1.83×10^{-1}
500	0.88	5.57	6.3×	5.22×10^{-1}	5.19×10^{-1}
1,000	1.04	5.77	5.5×	7.01×10^{-1}	7.00×10^{-1}
2,000	1.22	5.64	4.6×	9.30×10^{-1}	9.29×10^{-1}
5,000	1.34	6.30	4.7×	1.24	1.24
10,000	1.78	5.76	3.2×	4.39×10^{-10}	4.45×10^{-10}
25,000	2.00	5.52	2.8×	7.17×10^{-11}	7.13×10^{-11}
50,000	2.20	5.59	2.5×	2.70×10^{-11}	2.25×10^{-11}
100,000	2.35	6.32	2.7×	1.20×10^{-10}	1.21×10^{-10}

Table 10: Native vs browser wall-clock ms/step and final energy drift (Plummer sphere, GPU LBVH, leapfrog, $\theta = 0.75$). Native timing is the mean of tree build, force, and integration components after warmup. Browser timing is wall-clock from console-log timestamps, which includes event-loop scheduling overhead.

The constant browser wall-clock time indicates that per-step duration is dominated by a fixed overhead rather than GPU compute. Subtracting the native GPU time from the browser wall-clock yields a constant residual of approximately 4.3 ms/step across all N values. This fixed cost is attributable to the Emscripten asyncify mechanism, which yields control to the browser event loop between timesteps via `emscripten_sleep(0)`, incurring JavaScript-to-WebAssembly context switching and event-loop scheduling latency. Crucially, this overhead is additive and constant: it does not grow with N, meaning the sub-linear scaling behaviour observed in native execution is preserved in the browser.

Energy drift values are nearly identical between the two platforms at each N, with relative differences below 1% for $N \geq 10000$ and below 2% at smaller N. The small discrepancies at lower N are attributable to differences in floating-point intermediate rounding between the native and browser WebGPU driver paths, but both platforms produce the same qualitative energy evolution and the same order-of-magnitude drift at every N. This confirms that the browser WebGPU implementation executes the same GPU compute shaders with equivalent numerical fidelity.

Placeholder: Plot of ms/step vs N for native (blue, rising from 0.61 to 2.35) and browser wall-clock (orange, flat at 5.5–6.3). X-axis: N from 100 to 100,000 (log scale). Y-axis: ms/step from 0 to 8. Shaded region between the two lines represents the fixed 4.3ms event-loop overhead.

Figure 12: Native vs browser execution time per step. The browser wall-clock is dominated by a fixed 4.3 ms event-loop scheduling overhead (shaded region) from Emscripten asyncify, independent of N. The overhead factor decreases from $9.2\times$ at $N = 100$ to $2.7\times$ at $N = 100000$.

4.4.2 Practical Particle Count Limits

At the maximum tested particle count of $N = 100000$, the GPU LBVH leapfrog path achieves 2.35 ms per step, corresponding to approximately 425 timesteps per second. In interactive mode with rendering overhead, this translates to frame rates well above 60 FPS for N up to at least 50,000 particles on the Apple M2 GPU. The maximum storage buffer binding size reported by the adapter is 128 MB, which accommodates the BVH node array ($2N - 1$ nodes) and particle state buffers up to approximately $N = 2 \times 10^6$ before memory limits are reached, though this was not tested experimentally.

At $N = 100000$, the GPU scheduling overhead remains a small fraction of total step time: integration (kick/drift dispatches) accounts for only 0.04 ms regardless of N, confirming that compute dispatch latency does not become a bottleneck at the tested scale.

4.4.3 Seed Robustness

Three independent Plummer sphere realisations ($N = 5000$, seeds 42, 123, 256) show consistent timing (6.9 to 11.2 ms/step, with the higher value for seed 123 attributable to a less favourable initial particle distribution affecting tree traversal depth) and energy drift ranging from 0.64 to 1.24. The variation in drift reflects the sensitivity of finite-N relaxation to the specific particle configuration, confirming that energy evolution is initial-condition-dependent as expected for an N-body system.

4.5 Integrated Assessment

The results collectively address the three research questions:

1. **Scalability (RQ1)**: The GPU LBVH Barnes–Hut implementation scales sub-linearly with N, achieving only a 3.9× increase in step time over a 1000× increase in particle count. The tree-based approach outperforms direct $O(N^2)$ summation for $N \geq 2000$, with tree construction dominating total step time at 74–79%.
2. **Numerical quality (RQ2)**: Energy conservation analysis reveals that 32-bit GPU floating-point precision is the primary constraint on numerical fidelity, dominating the effects of opening angle and timestep for the configurations tested. The leapfrog integrator maintains stable orbits and conserves momentum, but quantitative energy drift comparisons between integrators are confounded by the different precision of their force computation paths (32-bit GPU vs 64-bit CPU).
3. **Platform feasibility (RQ3)**: WebGPU on the Apple M2 sustains interactive frame rates ($>$ 60 FPS) for up to at least $N = 50000$ particles in native mode. Browser execution via Emscripten adds a fixed overhead of approximately 4.3 ms per step from event-loop scheduling, reducing the overhead factor from 9.2× at $N = 100$ to 2.7× at $N = 100000$. Energy drift is numerically consistent across platforms, confirming that the browser WebGPU path executes equivalent GPU computation. The primary platform constraints are 32-bit GPU arithmetic precision and the fixed per-step browser scheduling cost, not buffer limits or compute throughput.

5) Discussion

5.1 Interpretation of Results

5.1.1 Scalability in Context

The observed scaling behaviour, a 3.9× increase in step time over a 1000× increase in N, is substantially flatter than the theoretical $O(N \log N)$ expectation. For $N \log N$ scaling, a factor of 1000 in N should produce approximately a $1000 \times \frac{\log(100000)}{\log(100)} \approx 2500$ factor increase in work, yet the wall-clock time increases by only 3.9×. This compression is explained by GPU parallelism: at small N, the GPU's compute units are underutilised, and dispatch overhead dominates; as N grows, occupancy improves and the parallel hardware absorbs the increased arithmetic cost. The scaling curve therefore reflects the convolution of algorithmic complexity with hardware utilisation, not pure algorithmic cost.

The tree-build fraction of 74–79% across all tested N indicates that the LBVH construction pipeline, rather than force evaluation, is the computational bottleneck. This is consistent with the seven-pass pipeline design: the bitonic sort alone requires $O(N \log^2 N)$ comparisons across multiple dispatches, and each pass incurs GPU synchronisation overhead. Future optimisation should therefore target the tree-build pipeline, particularly the sort phase, which is known to be amenable to radix-sort alternatives with better GPU scaling (Karras, 2012).

5.1.2 Numerical Fidelity

The energy conservation results present a nuanced picture. The most significant finding is that 32-bit floating-point precision, not the integration scheme or the opening angle, is the dominant constraint on numerical fidelity in the GPU execution path. This manifests in two ways: the two-body orbit shows dramatically different energy drift between the 32-bit GPU and 64-bit CPU force computation paths, and the Plummer sphere theta sweep shows no sensitivity to opening angle at $N = 5000$, suggesting that tree approximation error is below the 32-bit noise floor.

The Plummer sphere energy drift of $\Delta \frac{E}{|} E(0)| \approx 1.2$ after 1,000 steps represents genuine physical evolution (expansion of a finite-N system with approximate forces) rather than a numerical failure. The kinetic energy remains constant to high precision while the potential energy magnitude decreases, consistent with a system that is slightly super-virial due to the combination of finite-N sampling, monopole-only force approximation, and 32-bit accumulated round-off. Similar unbinding behaviour is commonly observed in N-body simulations with insufficiently matched softening and initial-condition scale lengths (Binney & Tremaine, 2008).

5.1.3 WebGPU as a Compute Platform

The results demonstrate that WebGPU delivers on the promise of portable GPU compute. The native-versus-browser comparison quantifies this directly: at $N = 100000$, the browser achieves 6.3 ms per step compared to 2.35 ms natively, an overhead factor of only 2.7×. Crucially, this overhead is a fixed per-step cost of approximately 4.3 ms attributable to Emscripten's asyncify event-loop yield, not a multiplicative penalty on GPU computation. The GPU executes the same compute shaders on the same hardware in both environments; the browser merely adds scheduling latency around each step.

This fixed-overhead model has an important implication for scalability: as N increases and the GPU compute time grows, the fixed browser overhead becomes a diminishing fraction of total step time. The overhead factor drops from 9.2× at $N = 100$ (where the GPU does almost no work) to 2.7× at $N = 100000$ (where GPU compute dominates). For scientifically meaningful particle counts, the browser platform approaches native throughput. Energy drift is numerically consistent across platforms to within 1% for $N \geq 10000$, confirming that the browser WebGPU implementation produces equivalent computational results.

The 32-bit precision constraint and the dispatch overhead at small N remain inherent to the current WebGPU specification rather than implementation limitations of a particular backend (Sengupta et al., 2025). The direct-versus-tree crossover at $N \approx 1500$ sets a practical lower bound on the particle count at which hierarchical force evaluation is beneficial on this platform.

5.2 Comparison with Published Results

Direct comparison with CUDA-based Barnes–Hut implementations is limited by differences in hardware generation, floating-point precision (64-bit in CUDA studies versus 32-bit here), tree topology (octree versus BVH), and opening criteria. Nevertheless, several structural comparisons are informative.

Burtscher and Pingali report CUDA Barnes–Hut performance of approximately 10 billion body-body interactions per second on an NVIDIA GTX 280 (Burtscher & Pingali, 2011). At $N = 100000$ with $\theta = 0.75$, this corresponds to roughly 10 ms per step on 2010-era hardware. The WebGPU implementation achieves 2.35 ms per step at the same N, which is faster in absolute terms but benefits from 16 years of hardware improvement (Apple M2 versus GTX 280). The comparison suggests that WebGPU achieves performance within the same order of magnitude as native CUDA when hardware generation is accounted for, validating the platform's computational viability.

Gaburov, Bédorf, and Portegies Zwart report tree-construction overhead of approximately 15–25% of total step time in their CUDA octree implementation (Gaburov et al., 2010), com-

pared to 74–79% in this work. The substantially higher tree-build fraction here reflects the cost of the fully on-device LBVH construction via bitonic sort, which trades construction efficiency for the elimination of CPU-GPU transfer. CUDA implementations that build the octree on the CPU and upload it achieve lower build fractions but incur PCIe transfer costs not present in the LBVH approach.

Nyland, Harris, and Prins report direct $O(N^2)$ GPU throughput of approximately 10–30 GFLOP/s for N-body problems on contemporary NVIDIA hardware (Nyland et al., 2009). The WebGPU direct-summation path in this work achieves approximately 5.0 ms per step at $N = 5000$ (equivalent to $\frac{N^2}{t} \approx 5 \times 10^9$ interactions per second), which is competitive given the integrated GPU architecture of the M2 and the 32-bit precision.

5.3 Limitations

The results should be interpreted in light of the following limitations, which bound the external validity of the findings.

Single hardware platform. All experiments were conducted on a single Apple M2 system. The WebGPU specification is cross-platform, but GPU performance characteristics (compute unit count, memory bandwidth, scheduling granularity) vary substantially across vendors (NVIDIA, AMD, Intel, Apple). The timing results and scaling behaviour reported here may not generalise to other hardware, particularly discrete GPUs with different memory hierarchies.

32-bit GPU precision. The WebGPU specification does not require 64-bit floating-point support in compute shaders, and the wgpu-native backend on Apple M2 provides only 32-bit arithmetic. All GPU-side force evaluation and integration therefore operates at single precision. As the results demonstrate, this is the dominant constraint on numerical quality, limiting the utility of energy drift as a quantitative diagnostic. The effect is most pronounced at small N (two-body) where round-off accumulates over many steps without the statistical averaging that occurs in large-N systems.

Monopole-only approximation. The force evaluation uses only the monopole term of the multipole expansion. Quadrupole and higher-order terms, which are standard in production codes such as GADGET-2 (Springel, 2005), would reduce the force approximation error at a given θ and potentially allow larger opening angles for the same accuracy target.

Fixed timestep. All experiments use a global fixed timestep. Adaptive individual timestepping, as implemented in GADGET-2 (Springel, 2005) and other production codes, would improve accuracy in dense regions while maintaining efficiency in diffuse regions. The absence of adaptive timestepping means that the timestep must be chosen conservatively for the densest region of the simulation, which may be unnecessarily small for the majority of particles.

Potential energy limited to $N \leq 5000$. Energy drift as a quantitative metric is available only for small N where direct pair summation is computationally feasible. For larger N, numerical quality is assessed indirectly through kinetic energy trends and momentum conservation, which are necessary but not sufficient indicators of force accuracy.

No multi-seed statistical analysis at large N. Seed robustness was tested at $N = 5000$ only. Timing and energy variability at larger N may differ due to different particle distributions affecting tree structure and traversal depth.

6) Future Work

The limitations identified in the discussion suggest several concrete directions for extending this work, each tied to specific performance or capability goals.

6.1 Adaptive Individual Timestepping

The fixed global timestep used in this implementation requires a conservative choice of Δt that accommodates the densest region of the simulation. Adaptive individual timestepping, as implemented in GADGET-2 (Springel, 2005), assigns each particle a timestep proportional to a local dynamical timescale (such as $\sqrt{\frac{\varepsilon}{|a_i|}}$) and synchronises particles using block timestepping. Implementing this in WebGPU compute shaders would require per-particle step bookkeeping in storage buffers and more complex dispatch logic, but would significantly improve accuracy in the dense cores of Plummer spheres and disk centres without penalising the timestep for particles in diffuse outer regions.

6.2 Cross-Browser and Cross-Device Benchmarking

The browser benchmarks in this work were conducted in a single Chromium instance on the Apple M2. Systematic benchmarking across Chrome and Firefox (once WebGPU is enabled in Firefox), and across integrated GPUs (Apple M2, Intel Arc), discrete GPUs (NVIDIA, AMD), and mobile devices, would extend the platform feasibility findings. Of particular interest is whether the fixed 4.3 ms per-step event-loop overhead observed here is browser-specific or consistent across implementations, and whether the overhead varies on devices with different GPU scheduling characteristics. Such testing would extend the findings of Sengupta et al. (Sengupta et al., 2025) to the specific case of hierarchical N-body simulation across diverse consumer hardware.

6.3 Higher-Order Multipole Expansion

Extending the monopole approximation to include quadrupole moments in the BVH nodes would reduce the force approximation error at a given θ, potentially allowing the use of larger opening angles (and correspondingly faster traversal) for the same accuracy target. The additional per-node storage (a symmetric 3×3 tensor per node) fits within WebGPU storage buffer limits, and the extra arithmetic per interaction is modest. This extension would bring the force accuracy closer to production codes and enable more meaningful comparison with GADGET-2′s multipole expansion (Springel, 2005).

6.4 64-Bit Precision When Available

The results demonstrate that 32-bit GPU arithmetic is the dominant constraint on numerical quality. As the WebGPU specification evolves, optional 64-bit floating-point support in

compute shaders (`f64`) may become available on supporting hardware. Benchmarking the precision-versus-performance trade-off with 64-bit force evaluation would determine whether the energy conservation improvement justifies the throughput reduction, and would enable direct integrator-quality comparisons (leapfrog vs Euler) that are currently confounded by the precision difference between GPU and CPU paths.

6.5 Multi-Component Galaxy Models

The rotating disk scenario (Scenario C) uses a simplified single-component model. Extending the initial conditions to include a central bulge, an exponential disk, and a dark matter halo, following standard multi-component galaxy models (Binney & Tremaine, 2008), would enable comparison with observational rotation curves and provide a more physically realistic testbed for the solver. The simulation architecture already supports heterogeneous particle masses and could accommodate multiple components with minimal code changes, primarily in the initial-condition generator.

7) Conclusion

This work presented a hierarchical N-body solver for galactic dynamics implemented entirely in WebGPU, featuring a fully GPU-resident Linear Bounding Volume Hierarchy (LBVH) constructed and traversed on-device each timestep. The implementation was evaluated across three benchmark scenarios (two-body orbit, Plummer sphere, and rotating exponential disk) with systematic parameter sweeps targeting three research questions: scalability, numerical quality, and platform feasibility. The findings are summarised below.

7.1 Scalability (RQ1)

The GPU LBVH Barnes–Hut implementation demonstrated sub-linear scaling with particle count, increasing from 0.61 ms per step at $N = 100$ to 2.35 ms per step at $N = 100000$, a 3.9 $\times$ increase in runtime over a 1000$\times$ increase in N. This scaling, substantially flatter than the theoretical $O(N \log N)$ due to GPU parallelism, confirms that hierarchical force evaluation on WebGPU is effective at the particle counts relevant to interactive galactic dynamics simulation. The tree-based approach outperformed direct $O(N^2)$ summation for $N \geq 2000$, with tree construction accounting for 74–79% of total step time and force evaluation for 18–20%.

7.2 Numerical Quality (RQ2)

Energy conservation analysis revealed that 32-bit GPU floating-point precision is the dominant constraint on numerical fidelity in the current implementation, exceeding the effects of opening angle θ and timestep Δt for the configurations tested. The symplectic leapfrog integrator maintained stable orbits and conserved momentum to high precision, consistent with its theoretical properties (Binney & Tremaine, 2008; Springel, 2005). However, quantitative energy drift comparisons between integrators were confounded by the different precision of their force computation paths (32-bit GPU BVH versus 64-bit CPU octree), indicating that a fair comparison would require either 64-bit GPU arithmetic or a 32-bit CPU reference path.

7.3 Platform Feasibility (RQ3)

WebGPU proved capable of sustaining interactive frame rates (above 60 FPS) for up to at least $N = 50000$ particles on an Apple M2 GPU, with headless throughput of 425 timesteps per second at $N = 100000$. Browser execution via Emscripten WebAssembly adds a fixed overhead of approximately 4.3 ms per step from event-loop scheduling, yielding an overhead factor of 2.7$\times$ at $N = 100000$ that diminishes as GPU compute time grows with N. Energy drift is numerically consistent across native and browser platforms to within 1% for $N \geq 10000$, confirming that the browser WebGPU implementation executes equivalent GPU computation. The overhead is additive and constant, not multiplicative: the sub-linear scaling behaviour

observed in native execution is preserved in the browser, validating WebGPU as a viable platform for browser-based scientific simulation. The primary platform constraints are 32-bit arithmetic precision and the fixed per-step browser scheduling cost at small N, rather than buffer size limits or compute throughput.

7.4 Contribution

This work demonstrates that WebGPU's compute shader model is sufficient for a non-trivial scientific computing workload: a complete Barnes–Hut N-body solver with parallel tree construction, hierarchical force evaluation, and symplectic integration. By building from a single codebase that targets both native desktop and browser environments, the implementation shows that accessible, interactive galactic dynamics simulation is achievable without specialised hardware or vendor-locked APIs. The performance characteristics reported here, together with the honest assessment of precision limitations, provide a reference point for future work on GPU-accelerated scientific computing in WebGPU as the specification and its implementations continue to mature.

References

Aarseth, S., Henon, M., & Wielen, R. (1974). A Comparison of Numerical Methods for the Study of Star Cluster Dynamics. *\aap*, *37*(1), 183–187.

Barnes, J., & Hut, P. (1986). A hierarchical O(N log N) force-calculation algorithm. *Nature*, *324*(6096), 446–449. https://doi.org/10.1038/324446a0

Batcher, K. E. (1968). Sorting Networks and Their Applications. *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, 307–314. https://doi.org/10.1145/1468075.1468121

Binney, J., & Tremaine, S. (2008). *Galactic Dynamics: Second Edition* (REV - Revised, 2). Princeton University Press. http://www.jstor.org/stable/j.ctvc778ff

Burtscher, M., & Pingali, K. (2011). An Efficient CUDA Implementation of the Tree-Based Barnes Hut n-Body Algorithm. *GPU Computing Gems Emerald Edition*, . https://doi.org/10.1016/B978-0-12-384988-5.00006-1

Congote, J., Segura, A., Kabongo, L., Moreno, A., Posada, J., & Ruiz, O. (2011). Interactive visualization of volumetric data with WebGL in real-time. *Proceedings of the 16th International Conference on 3d Web Technology*, 137–146. https://doi.org/10.1145/2010425.2010449

Fluke, C. J., Barnes, D. G., Barsdell, B. R., & Hassan, A. H. (2011). Astrophysical Supercomputing with GPUs: Critical Decisions for Early Adopters. *Publications of the Astronomical Society of Australia*, *28*(1), 15–27. https://doi.org/10.1071/as10019

Freeman, K. C. (1970). On the Disks of Spiral and S0 Galaxies. *The Astrophysical Journal*, *160*, 811. https://doi.org/10.1086/150474

Gaburov, E., Bédorf, J., & Portegies Zwart, S. (2010). Gravitational tree-code on graphics processing units: Implementation in CUDA. *Procedia Computer Science*, *324*, 1119–1127. https://doi.org/10.1016/j.procs.2010.04.124

GPU for the Web Working Group. (2026,). *WebGPU (GPU for the Web) Specification*.

Karras, T. (2012). Maximizing parallelism in the construction of BVHs, octrees, and k-d trees. *Proceedings of the Fourth ACM SIGGRAPH / Eurographics Conference on High-Performance Graphics*, 33–37.

Khronos Group. (2024,). *WebGPU Specification*.

Kreyszig, E. (2011). *Advanced Engineering Mathematics* (10th ed.). Wiley.

LF, G., & Rokhlin, V. (2001). A Fast Algorithm for Particle Simulation. *Journal of Computational Physics*, *73*, 325–348. https://doi.org/10.1016/0021-9991(87)90140-9

Morton, G. M. (1966). *A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing*.

Nyland, L., Harris, M., & Prins, J. (2009). Fast N-body simulation with CUDA. *GPU Gem, Vol. 3*, 677–695.

Owens, J. D., Luebke, D., Govindaraju, N., Harris, M., Krüger, J., Lefohn, A. E., & Purcell, T. J. (2007). A Survey of General-Purpose Computation on Graphics Hardware. *Computer Graphics Forum*, *26*(1), 80–113. https://doi.org/https://doi.org/10.1111/j.1467-8659.2007.01012.x

Owens, J., Houston, M., Luebke, D., Green, S., Stone, J., & Phillips, J. (2008). GPU computing. *Proceedings of the IEEE*, *96*, 879–899. https://doi.org/10.1109/JPROC.2008.917757

Plummer, H. C. (1911). On the Problem of Distribution in Globular Star Clusters. *Monthly Notices of the Royal Astronomical Society*, *71*(5), 460–470. https://doi.org/10.1093/mnras/71.5.460

Salmon, J. K., & Warren, M. S. (1994). Skeletons from the Treecode Closet. *Journal of Computational Physics*, *111*(1), 136–155. https://doi.org/https://doi.org/10.1006/jcph.1994.1050

Sengupta, S., Wu, N., Varvello, M., Jana, K., Chen, S., & Han, B. (2025). From WebGL to WebGPU: A Reality Check of Browser-Based GPU Acceleration. *Proceedings of the 2025 ACM Internet Measurement Conference*, 1018–1024. https://doi.org/10.1145/3730567.3764504

Springel, V. (2005). The cosmological simulation code gadget-2. *Monthly Notices of the Royal Astronomical Society*, *364*(4), 1105–1134. https://doi.org/10.1111/j.1365-2966.2005.09655.x

Sung, N.-J., Ma, J., Kim, T., Choi, Y.-j., Choi, M.-H., & Hong, M. (2025,). *Real-Time Cloth Simulation Using WebGPU: Evaluating Limits of High-Resolution*. https://arxiv.org/abs/2507.11794

Sutter, H. (2005). The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. *Dr. Dobb's Journal*, *30*(3), 202–210. http://www.gotw.ca/publications/concurrency-ddj.htm

Usta, Z. (2024). WEBGPU: A NEW GRAPHIC API FOR 3D WEBGIS APPLICATIONS. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 377–382. https://doi.org/10.5194/isprs-archives-XLVIII-4-W9-2024-377-2024

Verlet, L. (1967). Computer ``Experiments" on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules. *Physical Review*, *159*(1), 98–103. https://doi.org/10.1103/PhysRev.159.98

Wang, Q. (2021). A hybrid Fast Multipole Method for cosmological N-body simulations. *Research in Astronomy and Astrophysics*, *21*(1), 3. https://doi.org/10.1088/1674-4527/21/1/3

Zwart, S. F. P., Belleman, R. G., & Geldof, P. M. (2007). High-performance direct gravitational N-body simulations on graphics processing units. *New Astronomy*, *12*(8), 641–650. https://doi.org/https://doi.org/10.1016/j.newast.2007.05.004